

זיהוי כוונת שינוי ומימוש חסר בקוד חומרה באמצעות DiffIntent RTL

קורן חנס ונועה סבג

פרויקט מסכם בקורס למידה עמוקה
עיבוד שפה טבעית לקוד מקור באמצעות PyTorch

תקציר

מאגרי קוד חומרה המתוארים ב-RTL, רמת הפשטה המתארת העברת נתונים בין אוגרים בכל מחזור שעון, מתפתחים באמצעות אלפי שינויים קטנים. שורה חסרה באתחול, בתנאי הגנה או בפקודת assertion, כלומר בדיקה מובנית של תכונת חומרה, עלולה ליצור תקלה שקשה לזהות בסקירת קוד. בפרויקט בחנו שתי משימות על שינויי SystemVerilog, שפה לתיאור ולאומות חומרה, מהיסטוריית OpenTitan, פרויקט חומרה פתוח: סיווג כוונת השינוי לארבע קטגוריות, זיהוי שינוי חלקי שנוצר באמצעות השמטה מבוקרת של קוד. בנינו מערך נתונים של 5,500 דוגמאות והשוונו מודל TF-IDF, ייצוג המעניק משקל גבוה למונחים הבולטים בדוגמה אך נדירים במאגר, עם רגרסיה לוגיסטית, מסווג ליניארי הסתברותי, לשלושה מודלים עצביים שמומשו בספריית PyTorch. המודל ההיררכי מייצג בנפרד שורות שנוספו, נמחקו או נשמרו כהקשר. TF-IDF השיג Macro-F1 של 0.4197; מדד זה מחשב ממוצע שווה משקל של F1 בין המחלקות. המודל ההיררכי הוביל בזיהוי מימוש חסר, עם F1 של 0.6029, המאזן בין דיוק ההתראות לבין שיעור החסרים שנמצאו, ו-AUROC של 0.7352, המודד את יכולת הדירוג בכל ספי ההחלטה. הסרת הטמעת סוג השורה הורידה את F1 ל-0.4724. עם זאת, מודל המבוסס רק על הודעת השינוי הגיע ל-Macro-F1 של 0.9855, ממצא החושף תלות חזקה בין הודעות השינוי לבין התוויות החלשות. המסקנה היא שמבנה השינוי מועיל לזיהוי שלמות, אך הערכה אמינה של כוונת שינוי דורשת תוויות שאינן נגזרות מהודעות השינוי.

1. מבוא ומוטיבציה

סקירת שינויי RTL שונה מסיווג טקסט רגיל. הקלט הוא code diff, כלומר תיאור טקסטואלי של ההבדלים בין שתי גרסאות, ומשמעותו תלויה גם באסימוני הקוד וגם בתפקיד המבני של כל שורה: תוספת, מחיקה או הקשר שלא השתנה. הרצאות הקורס על נתונים מובנים מדגישות שהארכיטקטורה צריכה להטמיע הטיה אינדוקטיבית, כלומר הנחה מוקדמת על היחסים החשובים בקלט [10]. בפרויקט הנחה זו מתבטאת בהיררכיה של אסימונים בתוך שורות ושל שורות בתוך שינוי. שינוי קצר עשוי להיות תקין תחבירית ובכל זאת חסר מבחינה פונקציונלית, ולכן המודל מיועד לתעדף שינויים חשודים לסקירה. הוא אינו מחליף סימולציה, המריצה תרחישי קלט, או אימות פורמלי, הבודק תכונות מתמטיות בכל המצבים המתוארים במודל. הפלט הוא ציון סיכון סטטיסטי המסייע לבודק אנושי ואינו הוכחה לנכונות החומרה.

1.1 שאלות המחקר

- האם ניתן לסווג מתוך הפרש הקוד את כוונת השינוי: תיקון תקלה, שינוי תכונה או התנהגות, ארגון מחדש או ניקוי קוד, ושינוי תצורה או תזמון?
- האם ניתן להבחין בין שינוי שלם לבין מימוש חסר מלאכותי שנוצר מתוך שינוי אמיתי?
- האם ייצוג היררכי של הפרש הקוד וסוגי השורות משפר ביצועים לעומת ייצוגים לקסיקליים ורצפים שטוחים?

2. בסיס תאורטי ועבודות קשורות

2.1 מסגרת תאורטית

הפרויקט נשען על מסגרת הלמידה המונחית שנלמדה בקורס. כל מודל מקבל ייצוג של שינוי x , מפיק הסתברויות למחלקות ומעדכן את הפרמטרים באמצעות backpropagation, כדי למזער cross entropy [7,8]. ברגרסיה לוגיסטית הקשר בין הייצוג לפלט ליניארי, ואילו רשתות עמוקות לומדות שכבות ביניים של ייצוגים. משום כך ההשוואה בין המודלים בוחנת שתי שאלות נפרדות: איזה מידע נכלל בייצוג, ואיזה מבנה חישובי מתאים ליחסים שבתוכו.

המעבר מטקסט טבעי לקוד אינו משנה את עקרונות היסוד, אך הוא משנה את ההטיה האינדוקטיבית הנדרשת. הקורס מציג היררכיה ומקומיות כדוגמאות להתאמת מבנה המודל למבנה הנתונים [10]. כאן המקומיות מתקיימת בתוך שורת קוד, וההיררכיה מחברת בין אסימונים, שורות ומקטעי שינוי. חומר הקורס על עיבוד שפה טבעית מציג tokenization, n-grams, embeddings, ורשתות חוזרות לייצוג רצפים [11]. חומר ה-Transformers מוסיף attention ו-positional encoding, כדי ללמוד קשרים בין איברים רחוקים [12]. עקרונות אלה מופיעים בהמשך בתוך קדם-העיבוד, הארכיטקטורה וניסויי האבלציה, ולא כרשימת מונחים נפרדת.

הערכת מודל דורשת גם הבחנה בין התאמה לנתוני האימון לבין הכללה לדוגמאות שלא נראו. לכן הפרויקט משתמש בקבוצת אימות לבחירת נקודת העצירה, ב-dropout וב-weight decay להפחתת התאמת יתר, בהתאם לעקרונות הרגולריזציה והעצירה המוקדמת שנלמדו בקורס [9]. החלוקה הכרונולוגית והבדיקות נגד דליפת מידע נועדו להבטיח שהמדדים הסופיים משקפים ככל האפשר תחזית של שינויים עתידיים.

2.2 עבודות קשורות

מחקרים קודמים מראים כי ייצוג של שינוי קוד צריך לשמר את הפעולה שבוצעה ולא רק את תמונת המצב הסופית. CC2Vec לומד ייצוג של תיקון מתוך הקוד שנוסף, הקוד שנמחק והודעת השינוי PatchNet [2]. מארגן תיקון כמבנה היררכי [3], ו-DeepJIT משלב את הודעת השינוי עם הקוד לצורך חיזוי תקלות [4]. CodeReviewer מרחיב כיוון זה באמצעות קדם-אימון למשימות סקירת קוד [5]. הפרויקט מאמץ את עקרונות הייצוג המבני, אך מתאים אותו ל-RTL: סוג השורה מיוצג במפורש, והמקודד משותף לשני יעדים, כוונת השינוי ושלמות המימוש. אימון כזה נקרא למידה רב-משימתית, משום שאותם פרמטרים פנימיים מתעדכנים משתי פונקציות מטרה.

3. מערך הנתונים

3.1 כרייה וסינון

הדוגמאות נכרו אוטומטית מהיסטוריית [1] OpenTitan, פרויקט קוד פתוח לפיתוח רכיבי אבטחה בחומרה. נשמרו שינויים בקובצי Verilog, שפת תיאור חומרה שעליה מבוססת SystemVerilog. הוחרגו התיקיות DV המשמשות לאימות התכן, וכן out, tests, vendor, third_party, generated, build. בנוסף הוסרו שינויי מיזוג, פעולות revert המבטלות שינוי קודם, הודעות ריקות והפרשי קוד גדולים מדי. יחידת הדוגמה היא שינוי בקובץ יחיד. commit הוא רשומת שינוי שמורה במערכת Git, ו-hunk הוא מקטע רציף בתוך הפרש הקוד. commit שבו שונו עד שלושה קובצי RTL יכול לייצר כמה דוגמאות. התהליך הפיק 5,903 מועמדים גולמיים, ומתוכם 2,843 שינויים אמיתיים קיבלו תווית כוונה חד-משמעית.

3.2 תיוג כוונה

מאחר שבמאגר אין תוויות ידניות לכוונת השינוי, השתמשנו בשיטת weak labeling: התוויות נוצרות אוטומטית מכללים שנקבעו מראש, ולכן הן זולות להפקה אך עלולות להיות רועשות. ארבע התוויות הן bug_fix לתיקון תקלה, feature_or_behavior_change לשינוי תכונה או התנהגות, refactor_cleanup לארגון מחדש או ניקוי קוד, ו-configuration_timing לשינוי תצורה או תזמון. לדוגמה, fix או bug מצביעות על bug_fix, ואילו clock, אות התזמון המחזורי של המעגל, reset, אות המחזיר את המעגל למצב התחלתי, או parameter, קבוע תצורה בזמן בניית החומרה, מצביעות על configuration_timing. דוגמה ללא התאמה הוסרה, וכך גם דוגמה שבה כמה קטגוריות קיבלו ציון מרבי. כך הוסרו 2,713 דוגמאות ללא התאמה ו-347 דוגמאות עמומות. מגבלת השיטה מהותית: התוויות נגזרות מהודעת השינוי ולכן אינה בלתי תלויה באחד ממקורות הקלט האפשריים.

3.3 יצירת דוגמאות של מימוש חסר

כל שינוי אמיתי סומן כדוגמה שלמה. לאחר מכן נוצרה לכל היותר דוגמה חסרה אחת באמצעות מוטציה מלאכותית, כלומר שינוי מבוקר בקוד שנועד לדמות טעות מימוש. המוטציה הראשונה שניתן היה להחיל באזור שהשתנה נבחרה מתוך הרשימה הבאה: הסרת guard, תנאי המונע פעולה כאשר דרישות מוקדמות אינן מתקיימות; הסרת assertion, בדיקה של תכונה נדרשת; הסרת השמת reset או default, הקובעת מצב התחלתי או ערך ברירת מחדל; השמטת שורה שנוספה; השמטת hunk; החלפת אופרטור השוואה או לוגיקה; או היפוך קבוע בינארי פשוט. הדוגמה המקורית והדוגמה המוטנטית חולקות pair_id, מזהה צמד, ונשמרות באותה קבוצת נתונים. שמירת הצמד מונעת מן המודל לראות באימון את המקור שממנו נגזרה דוגמת בדיקה. סדר העדיפויות של המוטציות מסביר מדוע drop_added_line, השמטת שורה שנוספה, נפוצה הרבה יותר ממוטציות עדינות. טבלה 1: נתוני מערך הנתונים המלא.

מדד	ערך
דוגמאות גולמיות	5,903
שינויים אמיתיים מתויגים	2,843
דוגמאות לאחר יצירת מימושים חסרים	5,500
אימון / אימות / בדיקה	3,873 / 830 / 797
שינוי שלם / מימוש חסר מלאכותי	2,843 / 2,657
אורך ממוצע של הפרש הקוד	37.678 שורות מנורמלות

3.4 התפלגות וקדם-עיבוד

התפלגות הכוונות אינה מאוזנת: feature_or_behavior_change מהווה 40.7% מהדוגמאות, bug_fix מהווה 33.8%, configuration_timing מהווה 17.0%, ו-refactor_cleanup רק 8.5%. לעומת זאת, תוויות השלמות כמעט מאוזנות. הפרש הקוד מומר לשורות מסומנות: FILE מציין את הקובץ, HUNK מציין מקטע שינוי, CTX שורת הקשר, DEL שורה שנמחקה ו-ADD שורה שנוספה. בהתאם לחומר הקורס על עיבוד שפה טבעית [11], tokenization הופך את הקלט ליחידות שהמודל יכול לייצג. המפרק המותאם ל-SystemVerilog שומר מזהים, מילות מפתח, ערכים מילוליים, הוראות macro ואופרטורים. האורך מוגבל ל-128 שורות ול-64 אסימונים בשורה, ובקיצוץ נשמרות תחילה השורות שהשתנו וההקשר הקרוב. אוצר המילים נבנה רק מקבוצת האימון, כדי שאסימונים מקבוצות האימות והבדיקה לא ישפיעו על הייצוג.

3.5 חלוקת הנתונים והערכת דליפה

הדוגמאות מיונו כרונולוגית לפי תאריך השינוי וחולקו ביחס 15/15/70 לקבוצות אימון, אימות ובדיקה. קבוצת האימון משמשת להתאמת הפרמטרים, קבוצת האימות לבחירת המודל ונקודת העצירה, וקבוצת הבדיקה להערכה סופית בלבד. כל צמד של דוגמת מקור ומוטציה נשמר באותה קבוצה. כך נמנעה דליפה ישירה, מצב שבו מידע מדוגמת בדיקה משפיע על האימון ומנפח את המדדים. החלוקה אינה מופרדת לפי קבצים: אותו קובץ RTL עשוי להופיע בשינויים שונים ובקבוצות שונות. לכן התוצאות מודדות הכללה קדימה בזמן בתוך OpenTitan, ולא הכללה לקבצים או לפרויקטים שלא נראו באימון.

4. מודלים ושיטת האימון

4.1 המודלים שנבדקו

מודל הבסיס משתמש ב-TF-IDF, ייצוג דליל המעניק משקל גבוה למונחים תכופים בדוגמה אך נדירים במאגר, וברגרסיה לוגיסטית, מסווג ליניארי הסתברותי שנלמד במסגרת מודלי הסיווג בקורס [7]. הייצוג כולל מילים ו-character n-grams, הלוכדים גם חלקי מזהים ואופרטורים. MLP היא רשת הזנה קדימה רב-שכבתית [7]. BiGRU היא רשת חוזרת דו-כיוונית שבה שערים שולטים בזרימת המידע לאורך הרצף, ולכן היא מרחיבה את מודלי הרצף שנלמדו בהרצאת עיבוד השפה הטבעית [11]. המודל ההיררכי מקודד תחילה כל שורה ולאחר מכן את רצף השורות. משקלי מחלקות מגדילים את תרומת המחלקות הנדירות לפונקציית ההפסד, ו-dropout מאפסת באקראי חלק מן ההפעלות בזמן האימון כדי להפחית התאמת יתר [9].

טבלה 2: ארבעת המודלים המרכזיים.

מודל	ייצוג	הגדרות עיקריות
TF-IDF ורגרסיה לוגיסטית	מילים ו-char n-grams	עד 50 אלף מאפיינים ומשקלי מחלקות מאוזנים
MLP	הטמעת אסימונים ומיצוע	הטמעה 128, שכבה חבויה 256, dropout 0.2
BiGRU	רצף אסימונים שטוח	שכבה חבויה דו-כיוונית 128 וצבירת קשב
Hierarchical Transformer	אסימונים בתוך שורות ושורות בתוך שינוי	שתי שכבות, ארבעה ראשי קשב והטמעת סוג שורה

4.2 הארכיטקטורה ההיררכית

הארכיטקטורה ההיררכית מיישמת את רצף הרעיונות מהרצאות עיבוד השפה הטבעית וה-Transformers [11,12]. כל אסימון ממופה ל-embedding, וקטור נלמד המייצג אותו במרחב רציף. מקודד BiGRU קורא כל שורה בשני הכיוונים ומפיק וקטור שורה, כך שכל אסימון מושפע מן ההקשר שלפניו ואחריו. לוקטור מצורפת הטמעה נלמדת של סוג השורה, ולאחר היטל מתווספת positional encoding, המייצגת את מיקום השורה ברצף. מקודד Transformer מפעיל self-attention: כל שורה מקבלת משקל ביחס לשורות האחרות על סמך הרלוונטיות הנלמדת ביניהן. מנגנון צבירת קשב נוסף מסכם את רצף השורות לייצוג יחיד של השינוי. לבסוף שני ראשי סיווג, שכבות פלט נפרדות החולקות את אותו מקודד, מנבאים את הכוונה ואת שלמות המימוש.

איור 1: תרשים זרימה ניתן לעריכה של המודל ההיררכי.

שלב 1	שלב 2	שלב 3	שלב 4	שלב 5
אסימונים בשורה	BiGRU לכל שורה	הטמעת סוג שורה	Transformer בין שורות	שני ראשי סיווג

4.3 פונקציית המטרה

כל ראש מאומן באמצעות cross entropy, פונקציית ההפסד המקובלת לסיווג הסתברותי רב-מחלקתי או בינארי [7]. היא מקטינה את ההסתברות שהמודל מייחס לתשובה שגויה ומגדילה את ההסתברות לתווית הנכונה. באימון רב-משימתי שתי המשימות חולקות את אותו מקודד ומתעדכנות יחד; סכום ההפסדים הוא:

$$L = L_{intent} + \lambda L_{hole}$$

בניסוי המרכזי נקבע $\lambda = 1$, ולכן לשני ההפסדים משקל שווה. בניסויים החד-משימתיים חושב רק איבר ההפסד המתאים. נגזרות ההפסד חושבו באמצעות backpropagation [8], והפרמטרים עודכנו באמצעות AdamW. זהו ממטב ממשפחת Adam שמפריד את weight decay מעדכון הגרדיאנט. weight decay מענישה משקלים גדולים כחלק מן הרגולריזציה [9]. קיטום גרדיאנט בערך 1.0 מגביל את נורמת הגרדיאנט ומונע עדכונים חריגים. האימון נעצר לאחר שני סבבים ללא שיפור באימות, בהתאם לעקרון העצירה המוקדמת [9]. נקודת הביקורת, עותק שמור של פרמטרי המודל, נבחרה לפי Macro-F1 בכוונה, לפי F1 בשלמות, או לפי סכום שני המדדים באימון הרב-משימתי.

4.4 פרוטוקול ניסויי

ארבעת המודלים המרכזיים והמודל ללא סוגי שורות אומנו בשלושה אתחולים אקראיים שערכיהם 13, 42, 123. האתחול קובע את ערכי המשקלים הראשוניים ואת סדר הדוגמאות, ולכן חזרה בכמה אתחולים בודקת את יציבות התוצאה. ניסויי הסרת ההקשר, האימון החד-משימתי והמודל המבוסס על הודעה בלבד הורצו באתחול 13 בלבד, כדי לשמור על זמן ריצה סביר באמצעות Apple MPS, ממשק האצה למעבד הגרפי. במשימת הכוונה דווחו accuracy; Macro-F1, ממוצע שווה משקל של F1 בין המחלקות; Weighted-F1, ממוצע המשוקלל לפי שכיחות; ו-F1 לכל מחלקה. F1 הוא הממוצע ההרמוני של precision, שיעור ההתראות החיוביות הנכונות, ושל recall, שיעור המקרים החיוביים שנמצאו. במשימת השלמות דווחו גם AUROC, איכות הדירוג על פני כל ספי החלטה, ו-PR-AUC, השטח מתחת לעקומת precision-recall. חיובי במשימה זו הוא מימוש חסר. קבוצת הבדיקה לא שימשה לבחירת המודל.

4.5 ניסויי אבלציה

ניסויי אבלציה בודק את תרומתו של רכיב באמצעות הסרתו בלבד ושמירת יתר התנאים. זהו מבחן ישיר לשאלה אם ההטיה האינדוקטיבית שהוטמעה בארכיטקטורה אכן מועילה, ולא רק תיאור תאורטי שלה [10]. נבדקו ארבע גרסאות:

- ללא הטמעה של סוג השורה, לבדיקת תרומתו של מבנה הפרש הקוד.
- ללא שורות הקשר, לבדיקת תרומת הקוד הסמוך לשינוי.
- אימון נפרד לכל משימה, לבדיקת התרומה של למידה רב-משימתית.
- TF-IDF על הודעת השינוי בלבד, לבדיקת דליפת מידע מתהליך התיוג.

5. תוצאות

5.1 השוואת המודלים

טבלה 3: ממוצע תוצאות הבדיקה.

מודל	דיוק כוונה	Macro-F1 כוונה	דיוק חסר	F1 חסר	AUROC חסר
TF-IDF + LR	0.5132	0.4197	0.5734	0.5430	0.6123
MLP	0.4049	0.2961	0.5529	0.3293	0.5488
BiGRU	0.3998	0.3023	0.5822	0.4204	0.5942
Hierarchical Transformer	0.4065	0.3007	0.6725	0.6029	0.7352

5.2 סיווג כוונה

TF-IDF השיג את Macro-F1 הגבוה ביותר, 0.4197, לעומת 0.2961 עד 0.3023 במודלים העצביים. התוצאה מתיישבת עם תפקידם של מקטעי n-gram בייצוג טקסט [11]: במערך קטן יחסית, תבניות מילוליות מקומיות עשויות להספיק לזיהוי מילות מפתח, בלי ללמוד ייצוג עמוק. באתחול 13 הייתה feature_or_behavior_change המחלקה הקלה ביותר, ואילו refactor_cleanup הגיעה ל-F1 של 0.1642 ב-TF-IDF ול-0.0 במודל ההיררכי. מחלקה זו מהווה רק 8.5% מהנתונים, והניסוחים שלה חופפים לתיקוני תקלות ולשינויי התנהגות. המודל המבוסס על הודעת השינוי בלבד השיג Macro-F1 של 0.9855. מאחר שהתוויות נוצרו ממילות מפתח באותה הודעה, התוצאה אינה מעידה על הבנת קוד אלא על תלות בין מנגנון התיוג לקלט.

5.3 זיהוי מימוש חסר

במשימת השלמות התמונה מתהפכת. המודל ההיררכי השיג F1 של 0.6029 ו-AUROC של 0.7352, התוצאות הגבוהות ביותר מבין המודלים שנבדקו. באתחול 13 הפיק TF-IDF שליליים נכונים, 172 חיוביים שגויים, 168 שליליים שגויים ו-202 חיוביים נכונים. המודל ההיררכי הפיק 346, 81, 167, 203 בהתאמה. מספר המימושים החסרים שנמצאו כמעט זהה, אך המודל ההיררכי הפחית ביותר ממחצית את ההתראות השגויות על שינויים שלמים. זהו היתרון הצפוי מהטיה אינדוקטיבית המתאימה למבנה הנתונים [10]: ה-Transformer משווה בין שורות [12], וסוג השורה מסמן אם כל תוכן נוסף, נמחק או נותר כהקשר. השילוב מסייע להבחין בין שינוי קצר ושלם לבין שינוי שנקטע.

5.4 תוצאות ניסויי האבלציה

טבלה 4: תוצאות ניסויי האבלציה. כוכבית מסמנת מדד של ראש שלא אומן ולכן אינו משמש להשוואה.

הגדרה	מספר ריצות	Macro-F1 כוונה	F1 חסר	AUROC חסר
מודל היררכי מלא	3	0.3007	0.6029	0.7352
ללא סוג שורה	3	0.3028	0.4724	0.6294
ללא שורות הקשר	1	0.2677	0.5937	0.7126
אימון חסר בלבד	1	0.1813*	0.6354	0.7163
אימון כוונה בלבד	1	0.3193	0.5797*	0.4947*

הסרת סוגי השורות הורידה את F1 במשימת השלמות ב-0.1305 ואת AUROC ב-0.1058, כמעט ללא שינוי ב-Macro-F1 של הכוונה. באתחול 13 ירד מספר החיוביים הנכונים מ-203 ל-137, בעוד מספר החיוביים השגויים נשאר 81. האבלציה קושרת אפוא בין העיקרון התאורטי לתוצאה הניסויית: ייצוג מפורש של ADD, DEL ו-CTX מספק למודל מידע מבני שאינו מתקבל מרצף אסימונים בלבד [10]. לעומת זאת, הניסויים החד-משימתיים אינם מראים יתרון ברור ללמידה רב-משימתית, אך הם מבוססים על אתחול יחיד ולכן המסקנה מוגבלת.

6. ניתוח שגיאות ומגבלות

6.1 סוגי המוטציות

טבלה 5: שיעור הזיהוי לפי סוג מוטציה באתחול האקראי 13.

סוג מוטציה בקבוצת הבדיקה	מספר דוגמאות	שיעור הזיהוי של המודל ההיררכי
השמטת שורת ADD	304	0.6086
הסרת guard או assertion	58	0.2931
היפוך קבוע בינארי	6	0.0000
הסרת השמת reset או default	2	0.5000

82.2% מהדוגמאות החסרות בקבוצת הבדיקה נוצרו באמצעות השמטת שורת ADD. לכן המדד הכולל משקף בעיקר חסרים גלויים יחסית ואינו מודד באופן מאוזן את כל סוגי הטעות. המודל התקשה יותר בדוגמאות שבהן הוסרו guards או assertions, ונכשל בשש הדוגמאות של היפוך קבוע. בקבוצת הבדיקה לא הופיעו דוגמאות מסוג flip_logical_operator, flip_comparison_operator או drop_hunk, ולכן אין בסיס להערכת מוטציות אלה. המגבלה נובעת מתהליך יצירת הנתונים, ולא בהכרח מכשל של הארכיטקטורה בכל אחד מסוגי המוטציה.

6.2 דוגמאות איכותניות

חיוביים שגויים, כלומר שינויים שלמים שסומנו כחסרים, הופיעו בשינויים קטנים הקשורים לאות שעון, לאתחול או להסרת הגדרות תצורה תקינות. שליליים שגויים, כלומר מימושים חסרים שלא זוהו, הופיעו כאשר השמטת שורה לא פגעה בקוהרנטיות הלוקלית של ההפרש, למשל בשינוי של parameter או assertion. הדפוס מסביר את הפער בין למידת מבנה לבין הבנת התנהגות: המודל מזהה יחסים בין אסימונים ושורות, אך אינו מריץ את המעגל ואינו בודק את משמעותו לאורך זמן.

6.3 מחקרי הרחבה

הרחבת רשימות מילות המפתח הפחיתה את שיעור הדוגמאות שנפסלו בניסויי בדיקה מהירים, אך אינה פותרת הודעות המתארות מה השתנה במקום מדוע. מחקר ראשוני על Caliptra, פרויקט פתוח נוסף לחומרת אבטחה, מצא פרופיל שינויים שונה מאוד מזה של OpenTitan ושיעור פסילה גבוה יותר, ולכן אין להסיק ממנו הכללה בין פרויקטים. מודל Random Forest, אוסף של עצי החלטה שתחזיותיהם משולבות, עם 18 מאפייני RTL הגיע באימות צולב בן חמישה קפלים ל-Macro-F1 של 0.525 ± 0.114 , לעומת 0.053 ± 0.544 ב-TF-IDF באותו פרוטוקול. באימות צולב מחלקים את הנתונים לחמישה חלקים, מאמנים בכל פעם על ארבעה ובודקים על החלק החמישי; הסימון $\pm$ מציג ממוצע וסטיית תקן בין הקפלים. היחס בין מספר השורות שנוספו למספר השורות שנמחקו, גודל השינוי והופעת parameters היו המאפיינים הבולטים ביותר. תוצאות אלה משתמשות בנתונים ובפרוטוקולים אחרים ולכן אינן בנוות השוואה ישירה לטבלה 3.

7. מסקנות

הפרויקט מראה שמבנה השינוי מועיל בזיהוי מימוש חסר. המודל ההיררכי הפחית התראות שווא, והאבלציה אישרה שסוגי השורות הם רכיב משמעותי. בכך מתקבל רצף עקבי בין התאוריה לניסוי: הטיה אינדוקטיבית מתאימה מגדירה אילו יחסים המודל יכול לנצל [10], tokenization ו-embeddings מייצגים את הקוד [11], ו-attention עם positional encoding מחבר בין השורות [12]. לעומת זאת, במשימת הכוונה TF-IDF היה עדיף, והתוצאה הקיצונית של מודל ההודעה חשפה את מגבלת weak labeling. לכן אין להסיק שמודל עמוק עדיף בכל מצב. מחקר אמין יותר של כוונת שינוי דורש תת-קבוצה המתוגנת ידנית או תוויות ממקורות בלתי תלויים, כגון מערכת ניהול משימות ודיוני סקירת קוד. עבודה עתידית צריכה לאזן מוטציות עדינות, לבצע חלוקה ברמת הקובץ או תת-המערכת, להריץ את כל האבלציות בכמה אתחולים, ולשלב מידע תחבירי או סמנטי המותאם ל-RTL.

8. תרומת חברי הצוות

קורן חנס פיתח את תשתית הכרייה והעיבוד, את מנגנון יצירת הדוגמאות החסרות, את ארבעת המודלים ואת מערך האימון וההערכה, וביצע את ניסויי האבלציה ואת ההרצה המלאה. נועה סבג ביצעה את ניתוח השגיאות, הרחיבה את מנגנון התיג, חקרה את מאגר Caliptra, פיתחה מודל המבוסס על מאפייני RTL והדגימה כלי עזר לסקירת קוד. הניתוח והמסקנות הסופיים גובשו במשותף.

מקורות

- [1] lowRISC, OpenTitan, <https://github.com/lowRISC/opentitan>.
- [2] T. Hoang, H. J. Kang, J. Lawall, D. Lo, CC2Vec: Distributed Representations of Code Changes, ICSE 2020, DOI 10.1145/3377811.3380361.
- [3] T. Hoang, J. Lawall, Y. Tian, R. J. Oentaryo, D. Lo, PatchNet: Hierarchical Deep Learning-Based Stable Patch Identification for the Linux Kernel, arXiv:1911.03576, 2019.
- [4] T. Hoang, H. K. Dam, Y. Kamei, D. Lo, N. Ubayashi, DeepJIT: An End-to-End Deep Learning Framework for Just-in-Time Defect Prediction, MSR 2019, DOI 10.1109/MSR.2019.00016.
- [5] Z. Li et al., Automating Code Review Activities by Large-Scale Pre-training, arXiv:2203.09095, 2022.
- [6] K. Hannes, N. Sebbag, DiffIntent-RTL, <https://github.com/korrenhannes/diffintent-rtl>.
- [7] S. Maymon, Deep Learning, Lectures 2-3: Feedforward Networks, Output Units and Cost Functions, Tel Aviv University, 2026.
- [8] S. Maymon, Deep Learning, Lectures 4-6: Architecture Design, Optimization and Backpropagation, Tel Aviv University, 2026.
- [9] S. Maymon, Deep Learning, Lectures 7-9: Regularization, Early Stopping and Dropout, Tel Aviv University, 2026.
- [10] S. Maymon, Deep Learning, Lectures 9-10: Convolutional Neural Networks and Inductive Bias for Structured Data, Tel Aviv University, 2026.
- [11] S. Maymon, Deep Learning, Lecture 11: Natural Language Processing, Tokenization, n-grams, Embeddings and Recurrent Networks, Tel Aviv University, 2026.
- [12] S. Maymon, Deep Learning, Lecture 12: Attention, Positional Encoding and Transformers, Tel Aviv University, 2026.